



µLab worksheet • Deploying Spark on your own cluster

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • GIVE THE CLUSTER A FOLDER IT CAN READ • STEP 1

A container can only see the folders it was handed at the moment it was created. µLab 1 handed the cluster exactly one, `kube`, so that it had somewhere to write its credentials. There is no command that adds a second folder afterwards: the only way is to throw the container away and make it again. So make an empty folder now and hand it over now, while rebuilding is free.

macOS: check this before anything else.

If your shell forces a platform, every container in this round is emulated instruction by instruction. The cluster starts, runs slowly, and then stops answering, and nothing in the failure points at the cause. It must print an empty line:

```
echo "$DOCKER_DEFAULT_PLATFORM"
```

If it prints `linux/amd64`, find where it is set, delete that line from the file, and clear it from this shell as well. Editing the file alone is not enough: this terminal already has the value.

```
grep -rn DOCKER_DEFAULT_PLATFORM ~/.zshrc ~/.zprofile ~/.bash_profile ~/.bashrc
unset DOCKER_DEFAULT_PLATFORM
docker rmi -f rancher/k3s:v1.36.4-k3s1 alpine/kubect1
```

The last line throws away anything already pulled for the wrong architecture. All three images this round uses publish an Apple Silicon build, so nothing here needs emulating.

Measured on an M-series Mac: forced to `linux/amd64` the cluster's runtime stopped answering after about half an hour and never came back; native, the three pods were running in **35 seconds**.

1 ☐ Make the folder, then rebuild the cluster around it.

Work in the same folder you started the cluster in during µLab 1, the one with `kube` in it.

```
mkdir lab
```

Do not skip this line. If `lab` does not exist when the next command runs, Docker creates it for you and it comes out owned by `root`. Everything you try to write into it afterwards then fails with `Permission denied`, and the only cure is to delete it with `sudo` and start again.

Now replace the cluster.

The first line deletes the µLab 1 cluster and everything on it, including the whale. That is intended: nothing from µLab 1 is needed again.

```
docker rm -f k8s
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 -v "$PWD/kube:/output" -v "$PWD/lab:/lab" \
  -v k3s-images:/var/lib/rancher/k3s/agent/containerd \
  rancher/k3s:v1.36.4-k3s1 server --disable=traefik \
  --write-kubeconfig /output/config --write-kubeconfig-mode 644
```

This is the µLab 1 command with **two arguments added**. The table repeats the rest in one line each so that nothing on this sheet is typed unexplained.

<code>rm -f k8s</code>	Delete the container named <code>k8s</code> . The <code>-f</code> kills it first if it is still running.
<code>-d</code>	Detached: the cluster keeps running after your prompt comes back.
<code>--name k8s</code>	Give it a name, so later commands can say <code>k8s</code> instead of an id.
<code>--privileged</code>	Kubernetes runs containers, so it needs the kernel privileges a container is normally denied.
<code>--tmpfs /run</code>	A small in-memory filesystem. <code>k3s</code> writes sockets there and they must not survive a restart.
<code>--tmpfs /var/run</code>	The same, for the second conventional location.
<code>-p 6443:6443</code>	Publish the API port to your laptop, so a client outside the container can reach it.

<pre>-v "\$PWD/kube:/output" -v "\$PWD/lab:/lab" -rv k3s-images:/var/lib/... rancher/k3s:... server --disable=traefik --write-kubeconfig ...</pre>	Bind-mount: the kube folder here appears inside at /output , which is where the credentials file gets written. The new one. The lab folder here appears inside the cluster at /lab . It is empty, and it stays yours: a bind mount is the same folder seen twice, not a copy, so whatever you put in it later is visible inside at once. Also new. A named volume, not a folder of yours, holding the images the cluster downloads. Keeping it outside the container means throwing the cluster away no longer throws away the 535 MB with it: a rebuild then takes seconds instead of minutes. The image: a whole Kubernetes in one binary. Pinned to v1.36.4-k3s1 so every laptop runs the same version. Everything after the image name is handed to the program inside. server means be the cluster, not join one. Skip the bundled web router. Nothing here serves web traffic, and it takes time to start. Where to write the credentials file, and with which permissions. 644 makes it readable by you rather than only by root.
--	--

Then get a client. It is the µLab 1 line, unchanged.

Linux users who installed kubectl in µLab 1 keep using it and skip this box. Nothing about the client changes this round: it talks to the cluster over the network and never touches lab .

```
docker run --rm -it --network container:k8s \
-v "$PWD/kube:/kube:ro" -e KUBECONFIG=/kube/config \
--entrypoint sh alpine/kubectl
```

<pre>--rm -it --network container:k8s -v "\$PWD/kube:/kube:ro" -e KUBECONFIG=/kube/config --entrypoint sh alpine/kubectl</pre>	Disposable, and interactive: a shell you can type many commands into, deleted when you leave. Join the cluster container's network instead of getting one. That is why 127.0.0.1:6443 works in here. The credentials, read-only. A client has no business editing them. Which cluster to talk to. The path is the one inside this container, not on your laptop. Run a shell instead of the single kubectl the image is built for. The image: kubectl and a small Alpine base, from Docker Hub.
--	--

Check the cluster answers. The NAME is the new container's id, so it is **not** the one µLab 1 showed: this is a different cluster.

```
kubectl get nodes
```

NAME _____	STATUS _____
------------	--------------

2 • SOME DEFINITIONS

Everything this round asks you to explain can be answered from this block. It is worth reading once before step 2 rather than hunting through it afterwards.

image store	The place a container runtime keeps the images it can start. Your laptop's Docker has one. The cluster runs a different runtime, containerd, with a store of its own, and neither can read the other's files. They use the same names, which is what makes the confusion easy. Panel 4 lists both.
driver	The program that runs your Spark code. It decides what work exists, hands the pieces out, and collects the answers. There is exactly one per job, and when it stops the job is over.
executor	A process that does pieces of work for one driver. It is started for a job and exits with it. An executor holds a number of cores, and runs one piece of work per core at a time.
Spark master	The process that hands out the cluster's cores. It runs nothing itself. Drivers ask it for resources, it decides which machines will supply them, and it keeps a list of who is available. Not to be confused with the Kubernetes control plane, which hands out pods. This one hands out cores, and it is a pod itself.
Spark worker	A process that offers cores and memory to a master, and starts executors when told to. It announces itself to the master on startup, which is the line step 4 looks for. A worker is long-lived; the executors it starts are not.
standalone mode	Spark running its own master and workers, rather than asking another system for resources. It is Spark's simplest arrangement and the one deployed here. It exists because Spark predates Kubernetes and had to schedule for itself.
Service	A stable name for whichever pods currently carry a label. The one new Kubernetes object this round adds. Pods are replaced and each replacement gets a new address, so nothing can be configured to talk to a pod directly. A Service is the name that does not change: workers are told spark://spark-master:7077 once and that stays true across every replacement.
cluster DNS	The name server every pod is given, which answers for Service names. It is what turns spark-master into an address inside the cluster. It answers for pods that are behind a Service, and for nothing else , which is the trap panel 10 describes.

3 • CREATE THE MASTER AND THE WORKERS BY HAND • STEP 2

Spark's own cluster is three things: a master process that hands out cores, worker processes that supply them, and a name the workers can dial that keeps working when a pod is replaced. In Kubernetes terms that is **one Service and two Deployments**, and µLab 1 already made a Deployment with a single kubectl create command. Three objects, so three commands, typed in this order.

2 ☐ Create the name first, then the master, then the workers.

In the kubectl shell. The order matters: the master looks its own name up when it starts, so if the Service is not there yet it fails and restarts until it is.

```
kubectl create service clusterip spark-master --clusterip="None" --tcp=7077:7077
```

<pre>create service clusterip</pre>	Make a Service of the ordinary kind, reachable only from inside the cluster. Nothing here is served to the outside world.
-------------------------------------	--

spark-master Its name, and therefore **the name that will resolve**. It also becomes the label the Service looks for, app=spark-master, which is the label the next command happens to give its pods.
 --clusterip="None" Do not give the Service an address of its own. Make the name answer with the pod's own address instead, because the master has to bind to whatever its name resolves to, and it cannot bind to an address that belongs to no machine.
 --tcp=7077:7077 The port to carry, and the port on the pod to carry it to. 7077 is the one Spark listens on for submissions.

```
kubectl create deployment spark-master --image=spark:3.5.4-python3 \
  -- /opt/spark/bin/spark-class org.apache.spark.deploy.master.Master \
  --host spark-master
```

create deployment NAME The same command as kubectl create deployment whale ... in µLab 1. The name also becomes the pods' app label, which is what the Service just asked for.
 --image=spark:3.5.4-python3 Which image the pods run. Note there is no registry in the name, so **Docker Hub**, and the cluster will have to fetch it: panel 4 is about where from.
 -- End of kubectl's own arguments. Everything after it is the command to run inside the container, exactly as in µLab 1's Job.
 spark-class ...Master Run one named Java class and stay in the foreground, which is what a container needs. The image's own default would start something else.
 --host spark-master The master's own argument: which name to publish itself under. It must match what the workers dial, or they connect and are then sent somewhere they cannot reach.

```
kubectl create deployment spark-worker --image=spark:3.5.4-python3 --replicas=2 \
  -- /opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker \
  spark://spark-master:7077 --cores 1 --memory 1g
```

--replicas=2 How many worker pods to ask for. Before the --, so kubectl reads it and Spark never sees it.
 spark://spark-master:7077 Which master to report to. The Service name from the first command, so this stays true however often the master's pod is replaced.
 --cores 1 --memory 1g What one worker offers. Deliberately small, so the arithmetic is legible: the cluster has as many cores as it has worker pods.

Then watch the pods appear:

```
kubectl get pods
```

The pods will sit in ContainerCreating for a while. One to three minutes here, and longer on a shared connection. Nothing is wrong: the cluster is downloading a 535 MB image, and panel 4 is about why it has to. Once the master has it the workers start in seconds, because by then it is already in the store.

master pod	_____	READY	_____
worker pod	_____	READY	_____
worker pod	_____	seconds until all three	_____

4 • COMPARE THE CLUSTER'S IMAGE STORE WITH THE LAPTOP'S • STEP 3

The wait in step 2 was a download, and it is worth knowing what was downloaded and from where. The cluster runs its containers with containerd, not with your laptop's Docker, and the two keep entirely separate collections of images. Neither can read the other's. List both and compare.

3 ☐ List the images the cluster has, then the ones the laptop has.

Both lines are typed **on your laptop**, not in the kubectl shell. crictl exists only inside the cluster container, which is why the first line has to reach in there to run it.

First, what the cluster has.

```
docker exec k8s crictl images
```

IMAGE	TAG	IMAGE ID	SIZE
docker.io/library/spark	3.5.4-python3	5908cada5243a	535MB

docker exec k8s Run a command **inside** the already-running k8s container, rather than starting a new one.

crictl images The cluster's own tool for talking to containerd, and its way of saying *list the images you can start*. It exists only in there.

Now, what your laptop has.

```
docker image ls spark:3.5.4-python3
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

docker image ls NAME Docker's own list, narrowed to one name. A header with no rows under it is not an error: it means Docker has no image called that.

A header and nothing else is the expected answer, and it is the point of this step. Nothing on this sheet ever asked you to pull Spark, so your Docker has never seen it. The cluster fetched its own copy in step 2 and put it somewhere Docker cannot read.

If you do get a row, it is because you pulled that image yourself at some point, and it makes the same point more sharply: the `IMAGE ID` will match the cluster's to the character, and the cluster downloaded it again anyway. The two `SIZE` columns disagree even then, because they measure different things, the download and the unpacked copy.

cluster IMAGE ID _____ laptop: ID, or *none* _____

Now answer the first question.

Two programs on one laptop, both keeping images, and only one of them has this one. Say what would have to be true for the cluster to have used a copy your Docker already held, and why it is not.

Panel 2 defines **image store**, and the two listings above were produced by two different programs: read which command each one needed.

5 • READ THE MASTER'S LOG AND FIND THE WORKERS • STEP 4

Three pods running proves Kubernetes started three programs. It does not prove they found each other, and they are two separate things: Kubernetes knows nothing about Spark, and would report a worker shouting into the void as perfectly healthy. The master writes down every worker that reports in, so its log is where the answer is.

4 ☐ Read what the master has logged since it started.

In the `kubect` shell, `deploy/spark-master` means *the pod of that Deployment*, so the pod's name does not have to be typed.

```
kubect logs deploy/spark-master | grep Master:
```

logs	Print what a container has written to its output since it started. Nothing is running or changed by this.
deploy/spark-master	KIND / NAME. Given a Deployment, <code>kubect</code> picks one of its pods and says which.
grep Master:	Ordinary shell again: keep only the lines the master class wrote, and drop several hundred lines of startup from every other part of Spark.

The three lines that matter, as observed:

```
INFO Master: Starting Spark master at spark://spark-master:7077
INFO Master: Registering worker 10.42.0.2:43581 with 1 cores, 1024.0 MiB RAM
INFO Master: Registering worker 10.42.0.3:39957 with 1 cores, 1024.0 MiB RAM
```

The address in the first line is the Service name from panel 2, not a pod. The other two are the workers announcing themselves, one line each, with what they are offering. Your addresses will differ; the shape will not.

workers registered _____ cores each _____

cores in total _____ RAM each _____

Now answer the question this round is about.

Two numbers now agree: the worker Deployment was asked for `replicas: 2` and the master reports two workers. They are counted by different things and they are not the same number. Say which one is which, and which of the two would be wrong first if a worker pod were deleted.

Panel 2 defines both counters: what a **Spark master** keeps a list of, and what a **Service** stands in for. μ Lab 1 panel 9 is the other half, where a controller was shown replacing a deleted pod.

Solved means panels 1 to 5 are done: the lab folder unzipped and handed to the cluster, master and two workers all reading 1/1 Running, both image stores listed and compared, and the master's log naming both workers with the cores each one offered.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

6 • RUN A JOB ON THE CLUSTER YOU JUST BUILT

Everything so far shows Spark is **running**. This shows it **computes**. The job is the one that ships inside the image: throw darts at a square, count how many land inside a circle, and multiply. It needs no data, so nothing has to be copied anywhere, which makes it the cheapest possible proof that the master, the workers and a driver can all reach each other.

Typed in the `kubectl` shell. The driver runs **inside the master pod**, which looks like a detail and is not: panel 10 says what happens when it runs anywhere else.

```
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --conf spark.driver.host=spark-master \
  --class org.apache.spark.examples.SparkPi \
  local:///opt/spark/examples/jars/spark-examples_2.12-3.5.4.jar 100
```

<code>exec</code>	Run a command inside a container that is already running, the same idea as <code>docker exec</code> in step 3, one level in.
<code>--</code>	End of <code>kubectl</code> 's own arguments. Everything after it belongs to the command being run, so <code>--master</code> is read by Spark and not by <code>kubectl</code> .
<code>spark-submit</code>	Spark's launcher: start a driver, ask a master for cores, run the code, print the result, exit.
<code>--master spark://...:7077</code>	Which master to ask. The Service name and port from step 2.
<code>--conf spark.driver.host=...</code>	The one that is easy to leave out. It tells the driver which name to give the executors so they can call it back. Left out, the driver gives them the pod's own name, which nothing can resolve, and the job hangs for ever. Panel 9 is what that looks like.
<code>--class org.apache...SparkPi</code>	Which program inside the jar to run. A jar holds many; this names one.
<code>local: ///opt/spark/...jar</code>	Where the jar is. <code>local:</code> means <i>already present in the image on every machine</i> , so Spark ships nothing. The third slash is the root of that filesystem.
<code>100</code>	The program's own argument: how many slices to split the darts into. More slices, more pieces of work, and a more precise answer.

Several hundred lines of Spark's own logging scroll past, and the answer is at the very bottom. Look for the pair below: the first line is the scheduler saying the computation finished and how long it took, the second is the result. On a two-worker cluster the whole submission took about **8 seconds**, most of it starting a JVM.

```
26/09/21 07:46:35 INFO DAGScheduler: Job 0 finished: reduce at SparkPi.scala:38,
                                     took 2.272697 s
Pi is roughly 3.1427459142745913
```

The digits after the third will not match anyone else's, including a second run of your own, because the darts are thrown at random. Anything near 3.14 is a success.

If the screen stops moving and nothing like this appears, the job is waiting rather than working: panel 10 is what that means.

your value _____ seconds _____

7 • DELETE A WORKER POD AND WATCH THE MASTER NOTICE

`µLab 1` deleted a pod and watched a controller put it back. The same thing happens here, except that something else was relying on that pod, so there are now two reactions to watch instead of one, written down by two different programs.

```
kubectl get pods
kubectl delete pod <one of the two workers>
kubectl get pods
kubectl logs deploy/spark-master | grep Master: | tail -3
```

<code>delete pod</code>	Remove one pod. It removes nothing else: the Deployment that asked for it is untouched and still wants two.
<code> tail -3</code>	Shell: keep the last three lines, which are now the three most recent things the master wrote.

As observed, in this order:

```
INFO Master: Removing worker worker-...-10.42.0.5-33537 on 10.42.0.5:33537
INFO Master: Telling app of lost worker: worker-...-10.42.0.5-33537
INFO Master: Registering worker 10.42.0.9:40593 with 1 cores, 1024.0 MiB RAM
```

The replacement pod starts in seconds rather than minutes, because the image is now in the cluster's store. Write down the new pod's name and the new address in the master's log, then say which of the two put the worker back, the Kubernetes controller or the Spark master, and what the other one did instead.

8 • REBUILD THE SAME CLUSTER FROM A FILE

Three commands built this cluster and they worked. What they did not do is write anything down: the only record that the master needs `--host spark-master` is your shell history. Tear the three objects down and make them again from a file that says exactly the same thing.

```
kubectl delete deployment spark-master spark-worker
kubectl delete service spark-master
kubectl apply -f https://raw.githubusercontent.com/oesteban/\
isc-302-2026-2027-spark-lab/main/spark-as-typed.yaml
kubectl get pods
```

The address is one line, broken here only to fit the page.

<pre>delete deployment A B delete service apply -f https://...</pre>	Two names after one kind. Deleting a Deployment deletes the pods it was keeping alive, which is the loop from μLab 1 running in reverse. A different kind, so a separate command. The Service was never owned by either Deployment. Make the cluster match what that file describes. <code>-f</code> is for <i>file</i>, and a URL counts as one, so nothing is downloaded to your laptop.
--	--

Three objects again, from one command, and the pods are back in **seconds** rather than minutes: the image never left the cluster's store. Open the file in a browser and read it; its header carries the three commands it replaces, line for line.

seconds to three pods _____ objects created _____

Now answer what the file is actually for.

The file and the three commands produce the same cluster, so the file has bought you a record and nothing else yet. Name one thing a file can ask for that `kubectl create` has no option to express.

The flag tables in step 2 are the place to look: every argument `create` accepts is listed there, and μ Lab 3 opens by needing two things that are not.

9 • EXTRA MILE • HAND THE CLUSTER THE IMAGE INSTEAD OF LETTING IT DOWNLOAD IT

Step 2 waited while the cluster downloaded 535 MB that may already have been on the same disk. It does not have to: an image can be written out of one store and read into the other, over a pipe, with no network at all. Time it, and compare with the wait you recorded in step 2.

On your laptop. If `docker image ls` found nothing in panel 4, this needs `docker pull spark:3.5.4-python3` first, which is the download you are trying to avoid, so the payoff is for the **next** cluster rather than this one.

```
docker save spark:3.5.4-python3 | docker exec -i k8s ctr -n k8s.io images import -
```

<pre>docker save NAME docker exec -i k8s ctr -n k8s.io images import -</pre>	Write an image out as a plain stream of bytes. With no <code>-o</code> file it goes to standard output. Ordinary shell, nothing to do with Docker: send the left command's output straight into the right command's input, without ever writing a 1 GB file to disk. <code>-i</code> keeps the container's input open, which is what the pipe needs. There is no <code>-t</code> because nothing here is typed by hand. containerd's own tool, and the namespace Kubernetes reads. An image imported into any other namespace is invisible to it. Read an image stream and add it to the store. The <code>-</code> is the other end of the pipe.
--	---

seconds to import _____ seconds to download, step 2 _____

Sixteen laptops downloading the same 535 MB at once is the slowest thing in this room. Say what you would do differently if you were setting this lab up for a class.

10 • ANNEX • WHY THE DRIVER HAS TO RUN SOMEWHERE THAT HAS A NAME

Leave `--conf spark.driver.host=spark-master` out of panel 6 and the job never starts. Executors are told to call the driver back by name, and the name they are given is the master pod's own, something like `spark-master-cc5688b64-1t59r`. Cluster DNS answers for Service names, not for pod names, so every executor fails to resolve it and exits, and the driver waits for cores that never arrive, printing this every fifteen seconds:

```
WARN TaskSchedulerImpl: Initial job has not accepted any resources; check your
cluster UI to ensure that workers are registered and have sufficient resources
```

The message blames the workers, and the workers are fine. The flag is a workaround for something `kubect1 create` cannot do: give a pod a name of its own in cluster DNS. `µLab 3` fixes it properly, and the flag is not needed again.

What production does instead. Standalone mode is Spark scheduling for itself, from before Kubernetes existed. Given a real cluster, a submission made with `--master k8s://...` skips all of this: Kubernetes is asked directly for a driver pod and executor pods, they exist only while the job runs, and there is no master, no workers and nothing standing idle in between. It is also several more things to get right, which is why it is here and not in step 2.

11 • ANNEX • WHAT THESE NUMBERS LOOKED LIKE HERE

Every figure below was measured on the teaching machine, on a fast connection, while this sheet was being written. **Yours will not match**, and they are not meant to: they are here so you can tell a slow laptop from a broken one. An order of magnitude out is worth asking about; a factor of two is not.

what	measured	where
cluster downloads the Spark image	1 m 9 s	step 2
the same download, slower connection	2 m 53 s	step 2
all three pods 1/1 Running	112 s to 182 s	step 2
workers restart once the image is in the store	about 20 s	panel 7
SparkPi on two workers	8 s	panel 6
handing the image over instead of downloading	37 s	panel 9
the image, as the cluster reports it	534'909'833 B	panel 4

Write your own beside them as you go. The last two rows are the interesting pair: the import moves the same bytes as the download and does it without a network, which is the whole argument of panel 9.

Done when the referee can say, without reading this sheet, why the cluster downloaded an image the laptop already had, and what a Service is for.